Module 7: LLMs for Search, Prediction, and Generation

Course Overview

This module explores the practical application of Language Models in three critical areas: search query completion, next word prediction, and text generation. These are among the most widely deployed applications of LLMs in industry, powering search engines, autocomplete systems, chatbots, and content generation platforms. Students will understand how embeddings, transformers, and attention mechanisms work together to enable intelligent search, accurate predictions, and coherent text generation. This module bridges foundational NLP concepts with real-world implementations at scale [1] [2].

Table of Contents

1. Introduction to Search, Prediction, and Generation
2. Word Embeddings and Semantic Representations
3. Search Query Completion Systems
4. Next Word Prediction and Language Modeling
5. Transformer Architecture: Deep Dive
6. Multi-Head Attention Mechanisms
7. Stacking Attention Layers
8. Text Generation with Transformers
9. Challenges and Optimization Techniques
10. Hands-on Implementation Guide

1. Introduction to Search, Prediction, and Generation

1.1 Three Pillars of LLM Applications

Search: Finding relevant information from vast collections of documents or queries. Modern search has evolved from keyword matching to semantic understanding [3].

Prediction: Forecasting what comes next—the next word in a sentence, the next query a user will type, or the next action. This is fundamental to both understanding and generation [4].

Generation: Creating new text based on context—from query suggestions to article writing to code generation. All LLMs are fundamentally generative models.

1.2 Interconnections Between These Pillars

These three pillars are deeply interconnected:

- **Search enables prediction:** Understanding what users search for helps predict their next query
- **Prediction enables generation:** Predicting the next token is the mechanism for text generation
- **Generation enables search:** Generated summaries, embeddings, and representations improve search

All three depend on shared foundations: embeddings, attention mechanisms, and transformer architectures.

1.3 Real-World Applications and Scale

Search Query Completion:

- Google Search: ~8.5 billion searches daily, autocomplete on 30% of queries
- E-commerce: Amazon, Alibaba use neural ranking for query suggestions
- Academic Search: Google Scholar, arXiv use semantic search

Next Word Prediction:

- Mobile keyboards (SwiftKey, Gboard): Real-time prediction with latency <100ms
- Email composition: Gmail Smart Compose, Outlook Suggestions
- Code completion: GitHub Copilot uses transformers for next-token prediction

Text Generation:

- ChatGPT: 100M users generating diverse content (language translation, code, creative writing)
- Content creation: News generation, product descriptions, social media posts
- Code generation: GitHub Copilot, Amazon CodeWhisperer
- Enterprise: Summarization, report writing, customer service automation

2. Word Embeddings and Semantic Representations

2.1 The Problem with Discrete Representations

Traditional NLP represents words as one-hot vectors—a vector with single 1 and rest 0s:

$$\text{word} = [0, 0, 0, 1, 0, \ldots, 0]$$

Limitations:

- Orthogonal (no similarity): "dog" and "cat" appear equally different
- High-dimensional: Vocabulary of 50k words = 50k dimensions
- Sparse: Almost all zeros, inefficient computation
- Cannot capture semantic relationships

2.2 Dense Word Embeddings

Word embeddings map words to dense vectors in continuous space (typically 50-300 dimensions) where similar words are close together[5].

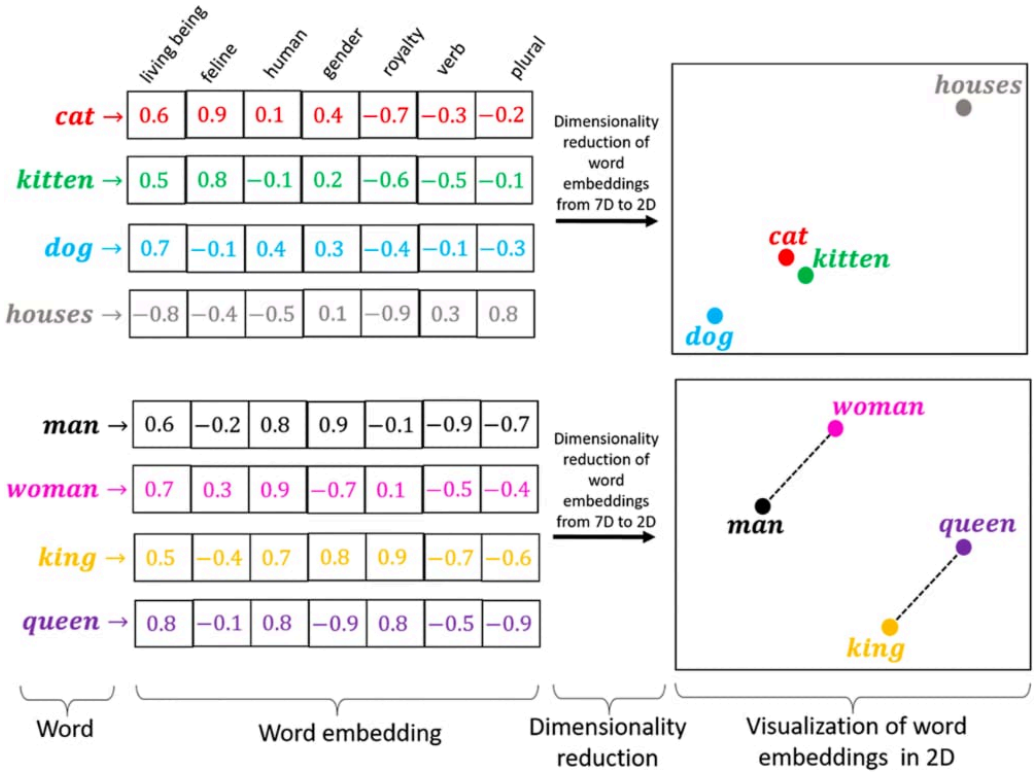

Figure 1: Word Embeddings: Words with similar meanings are positioned close in vector space

Key Properties:

- **Density:** All dimensions meaningful
- **Dimensionality:** Reduced to 100-300 dims (vs 50k one-hot)
- **Semantic Relationships:** Similar words cluster together
- **Compositionality:** "King" - "Man" + "Woman" ≈ "Queen"[6]

2.3 Word2Vec: Skip-gram and CBOW

Word2Vec learns embeddings by predicting context from words (Skip-gram) or words from context (CBOW).

Skip-gram Model:

Given word w_t, predict surrounding words $w_{t-c}, \ldots, w_{t+c}$ within context window c:

$$L = -\sum_{t=1}^{T} \sum_{-c \le j \le c, j \ne 0} \log P(w_{t+j} | w_t)$$

$$P(w_{t+j} | w_t) = \frac{\exp(v'_{w_{t+j}} \cdot v_{w_t})}{\sum_w \exp(v'_w \cdot v_{w_t})}$$

Where v_{w_t} is embedding of input word and $v'_{w_{t+j}}$ is embedding of context word.

Computational Optimization:

Computing softmax over full vocabulary (50k-100k words) is expensive. Skip-gram uses:

1. **Hierarchical Softmax:** Binary tree speeds up to $O(\log V)$
2. **Negative Sampling:** Sample negative words instead of computing full softmax

Negative Sampling:

Instead of full softmax, train to distinguish target from random negatives:

$$L = \log \sigma(v'_{w+} \cdot v_w) + \sum_{i=1}^{k} \mathbb{E}_{w^- \sim P_n}[\log \sigma(-v'_{w^-} \cdot v_w)]$$

Where w^+ is positive context, w^- are negative samples, k is number of negatives (5-15 typical).

2.4 GloVe: Global Vectors for Word Representation

GloVe combines local context windows (like Word2Vec) with global corpus statistics[7]:

$$J = \sum_{i,j=1}^{V} f(X_{ij})(v_i \cdot v_j + b_i + b_j - \log X_{ij})^2$$

Where:

- X_{ij}: Co-occurrence count of words i and j
- v_i, v_j: Word vectors
- $f(X_{ij})$: Weighting function preventing rare co-occurrences from dominating

Advantages over Word2Vec:

- Uses full corpus statistics (not just local context)
- Better captures global semantic relationships
- Faster convergence

2.5 Contextual Embeddings: Beyond Static Words

Static embeddings (Word2Vec, GloVe) assign single embedding per word, ignoring context.

Problem: "Bank" has different meaning in "river bank" vs "savings bank" but gets single embedding.

Solution: Contextual Embeddings

Context-dependent representations computed from full sentence:

- **ELMo:** Bidirectional LSTM layers, different representation per layer
- **BERT:** Bidirectional transformers, attend to all words simultaneously
- **GPT:** Unidirectional transformers (left-to-right)
- **ALBERT:** Parameter-efficient version of BERT

BERT Embeddings Example:

$$\text{embed}(\text{"bank"}, \text{"river bank"}) \neq \text{embed}(\text{"bank"}, \text{"savings bank"})$$

Computed from context, same word gets different embeddings in different sentences.

2.6 Embedding Arithmetic and Analogy Tasks

Embeddings capture semantic relationships enabling vector arithmetic[8]:

$$\text{embedding}(\text{"king"}) - \text{embedding}(\text{"man"}) + \text{embedding}(\text{"woman"}) \approx \text{embedding}(\text{"queen"})$$

Common Analogies:

- Occupational: "doctor" - "man" + "woman" ≈ "nurse"
- Geographic: "Paris" - "France" + "Germany" ≈ "Berlin"
- Relational: "bad" - "good" + "good" ≈ ... (contextual)

Mathematical Foundation:

The embedding space encodes linear relationships due to training objectives:

$$P(\text{queen}|\text{king}, \text{man}, \text{woman}) \propto \text{similarity to "queen"}$$

This works because neural networks learn linear transformations as primary pattern.

3. Search Query Completion Systems

3.1 Query Autocomplete Overview and Challenges

Query autocomplete (QAC) suggests queries as users type. Used by Google, Bing, Amazon, and all major search platforms [9].

Challenges:

- **Latency:** Suggest in <50ms (user typing speed is ~100-200ms per character)
- **Diversity:** Balance popularity, personalization, and relevance
- **Coverage:** Handle new and rare queries not in history
- **Freshness:** Reflect trending topics and seasonal changes
- **Privacy:** Avoid suggesting sensitive queries
- **Scale:** Billions of queries daily across millions of users

3.2 Traditional Approaches

Most Popular Completion (MPC):

Find queries starting with typed prefix, rank by frequency:

1. Index all queries in trie (prefix tree)
2. User types "python", traverse trie to "python" node
3. Return most-visited queries under that node

Advantages:

- Simple, extremely fast O(1) lookup
- No training required

Disadvantages:

- Ignores user intent and context
- Biased toward globally popular queries (not personalized)
- New queries not in index

Linear Ranking with Hand-Crafted Features:

Retrieve candidates via MPC, re-rank using linear scorer:

$$\text{score}(q) = w_1 \times \text{popularity}(q) + w_2 \times \text{CTR}(q) + w_3 \times \text{recency}(q)$$

Where weights w_i tuned on held-out data.

3.3 Neural Query Autocomplete

Neural approaches learn non-linear combinations of features, dramatically improving relevance [10].

Architecture:

Generative AI Project Lifecycle

	Training Duration	Customisation	Objective	Expertise Required
Pre-training a new LLM	Days/ Weeks/ Months	• Architecture • Size • Vocabulary • Context Window • Training Data	Next token preditction	High
Prompt Engineering	Not required	• Only prompt customisation	Increase task performance	Low
Fine tuning	Minutes/ Hours	• Task specific tuning • Domain specific data • Update model / adapter weights	Increase task performance	Medium
Reinforcement Learning from Human/AI feedback	Minutes/ Hours + data collection for reward model	• Train reward model [HHH goals] • Update model / adapter weights	Increase alignment with human preferences	Medium - High
Compression/ Optimisation/ Deployment	Minutes/ Hours	• Reduce model size • Faster inference	Increase inference performance	Medium

@abhinav_kimothi \in\abhinav-kimothi

Figure 2: Neural Query Autocomplete Pipeline: From prefix to ranking to display

Pipeline:

1. **Candidate Generation:** Retrieve ~100-1000 queries starting with prefix using trie/index
2. **Feature Engineering:** Extract features for each candidate
   - Query statistics: popularity, CTR, freshness, diversity
   - User signals: session context, device, location
   - Textual features: prefix overlap, query length, spelling
3. **Neural Ranking:** Score candidates with neural network
4. **Final Ranking:** Re-rank based on scores, diversity, and constraints
5. **Display:** Show top-k suggestions

Neural Ranking Model:

Simple feed-forward ranker

```
model = tf.keras.Sequential([
tf.keras.layers.Dense(256, activation='relu'),
tf.keras.layers.Dropout(0.3),
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dropout(0.2),
```

```
tf.keras.layers.Dense(1, activation='sigmoid') # Probability of relevance
])
```

Features for Each Candidate Query:

Feature Type	amp; **Feature Name**	amp; **Description**
Popularity	amp; Global frequency amp; Session frequency amp; CTR	amp; How often query submitted across all users amp; How often in current session amp; Click-through rate on search results
Temporal	amp; Recency amp; Seasonality	amp; Freshness (recent queries ranked higher) amp; Captures trends (e.g., "tax deadline" in April)
Textual	amp; Prefix match score amp; Edit distance	amp; Overlap with typed prefix amp; Typo tolerance (e.g., "pyton" → "python")
Contextual	amp; Department affinity amp; Device type amp; Geographic location	amp; User's typical search category amp; Mobile vs desktop (affects UI) amp; Location-relevant suggestions

Table 1: Neural QAC Feature Categories

3.4 Training Data and Learning-to-Rank

Learning-to-Rank (LTR) for QAC:

Instead of predicting relevance directly, rank candidate pairs:

Where query q_i clicked more than q_j for same prefix.

Data Collection:

- Positive examples: Queries users clicked on
- Negative examples: Queries shown but not clicked
- Implicit feedback: Click behavior indicates relevance

Bias in Training Data:

Presentation bias: Suggestions shown higher in list get clicked more (not necessarily better).

Mitigation:

- Use queries from sessions with autocomplete disabled (unbiased)
- Synthetic prefixes: Generate prefixes from complete queries
- Position-aware ranking: Account for presentation position

3.5 Production Deployment of QAC

Latency Budget:

- Total latency: <50ms user-visible
  - Index lookup: <5ms (trie traversal)
  - Feature engineering: <10ms (vectorization)
  - Neural scoring: <20ms (model inference)
  - Re-ranking/display: <10ms (sorting, formatting)

Scaling Strategies:

- **Caching:** Pre-compute scores for common prefixes
- **Quantization:** 4-bit model for faster inference
- **Batching:** Group requests across users
- **Early Exit:** Stop if confident suggestion after few candidates
- **Async Update:** Update trie asynchronously to not block queries

4. Next Word Prediction and Language Modeling

4.1 Language Modeling Fundamentals

A language model assigns probability to sequences of words:

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i | w_1, \ldots, w_{i-1})$$

Applications:

- **Text generation:** Sample from model to generate text
- **Ranking:** Score text (higher prob = better quality)
- **Correction:** Identify low-prob text (errors)
- **Speech recognition:** Disambiguate homophones based on prob

4.2 Next Token Prediction: The Core Mechanism

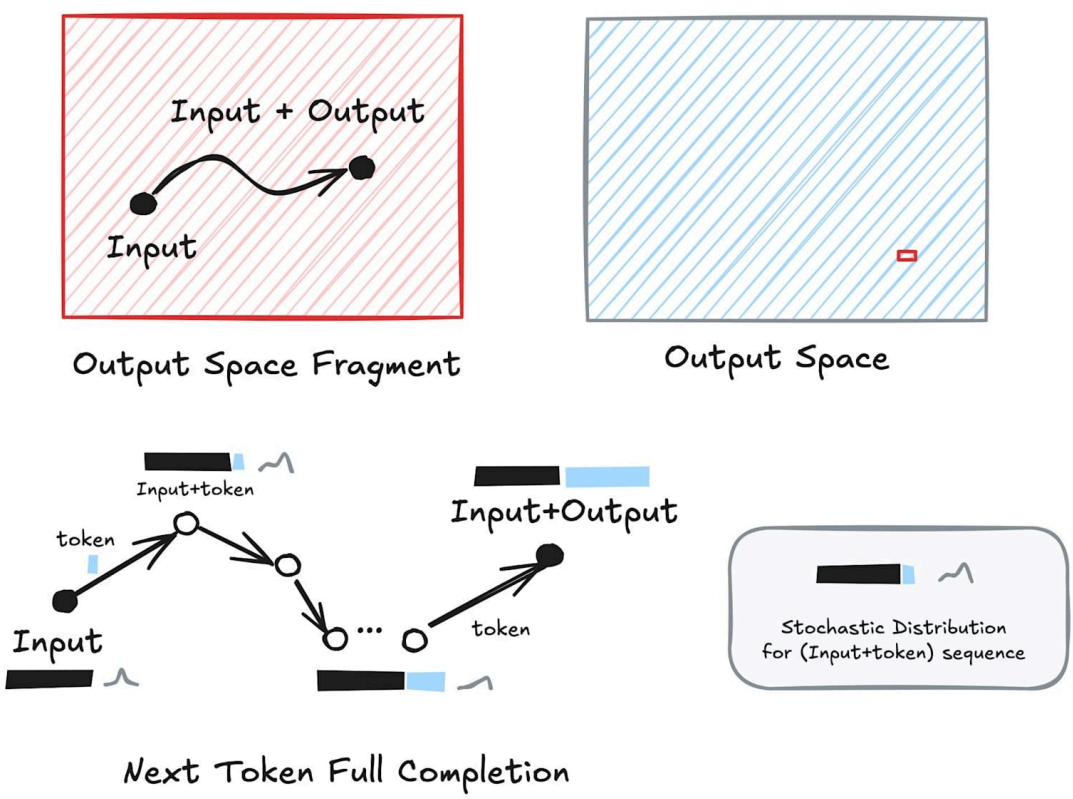

Figure 3: Next Token Prediction: Model computes probability distribution over vocabulary

Given context $h = [w_1, w_2, \ldots, w_{t-1}]$, predict probability of next token:

$$P(w_t | h) = \text{softmax}(f(h)W + b)$$

Where:

- $f(h)$: Representation of context (from transformer)
- W: Output embedding matrix (vocab_size × hidden_dim)
- b: Output bias

Sampling Strategies:

1. **Greedy:** Take highest probability token
   $w_t = \arg\max_w P(w_t|h)$
   - Deterministic, reproducible
   - Often leads to repetition
2. **Top-k Sampling:** Sample from top-k highest probability tokens
   - Reduces "silly" outputs
   - More diverse than greedy
   - Typical: k=40-50
3. **Nucleus (Top-p) Sampling:** Sample from smallest set of tokens with cumulative prob > p
   - Adaptive: focuses on likely tokens but allows tail probability
   - Better quality, more natural
   - Typical: p=0.9-0.95
4. **Temperature Scaling:** Adjust probabilities before sampling
   $P_{\text{scaled}}(w) = \frac{\exp(\log P(w)/T)}{\sum_w \exp(\log P(w)/T)}$
   - : Flatter distribution (more randomness)
   - : Sharper distribution (more greedy)
   - Typical: $T = 0.7 - 1.0$

4.3 RNN-Based Language Models

Basic Recurrent Language Model:

Update hidden state with each token:

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b_h)$$
$$P(w_t|h_t) = \text{softmax}(W_{ho}h_t + b_o)$$

Limitations:

- Vanishing gradients: Hard to capture long-range dependencies
- Slow training: Sequential processing, can't parallelize
- Limited context: Context compressed into single hidden state

4.4 Transformer-Based Language Models

Transformers overcome RNN limitations:

- **Parallelizable:** All positions computed in parallel
- **Long-range:** Attention attends to full history
- **Efficient:** Layer normalization and other techniques stabilize training

Decoder-only transformer (used by GPT, LLaMA):

- Compute representation for each position using self-attention
- Causal masking: Position t only attends to positions

- Output layer computes probability distribution

$$P(w_t|w_1, \ldots, w_{t-1}) = \text{softmax}(h_t W_o)$$

4.5 Training Language Models

Objective: Maximum Likelihood

Minimize negative log-likelihood:

$$L = -\sum_{i=1}^{n} \log P(w_{i+1}|w_1, \ldots, w_i)$$

Training Procedure:

1. Tokenize text
2. Create sequences of context-target pairs
3. Forward pass: encode context, compute next-token probabilities
4. Compute loss: cross-entropy between predicted and target
5. Backward pass: update parameters

Batching:

Process multiple sequences in parallel:

Example batch of sequences

batch = [
"The quick brown fox jumps over",
"Python is a programming language",
"Machine learning is transforming"
]

Targets: next word for each

targets = ["the", "used", "industries"]

4.6 Evaluating Language Models

Perplexity:

Exponentiated average negative log-likelihood:

$$\text{PPL} = \exp\left(-\frac{1}{n}\sum_{i=1}^{n} \log P(w_i|h_i)\right)$$

- Lower perplexity = better model
- Interpretation: Average branching factor (model thinks it could be ~PPL equally likely tokens)

Example:

- Random model over 50k vocabulary: PPL ≈ 50k
- Reasonable language model: PPL ≈ 50-100
- State-of-the-art: PPL ≈ 10-20

Downstream Task Evaluation:

- BLEU: For translation/summarization
- ROUGE: For summarization
- Exact match / F1: For QA

- Human evaluation: Fluency, coherence, relevance

5. Transformer Architecture: Deep Dive

5.1 Why Transformers Revolutionized NLP

The "Attention Is All You Need" paper (2017) introduced transformers, replacing RNNs[^11].

Key Innovation: Self-attention mechanism instead of recurrence.

Advantages:

- **Parallelizable:** All positions computed simultaneously (vs sequential in RNN)
- **Long-range:** Attention directly connects any two positions (no sequential bottleneck)
- **Efficient pre-training:** Can train on massive datasets
- **Transfer learning:** Pre-trained models transfer well to downstream tasks

Timeline:

- 2017: Original Transformer (Vaswani et al.)
- 2018: BERT (bidirectional)
- 2018: GPT (unidirectional, language modeling)
- 2019-2024: Scaling to trillions of parameters

5.2 Transformer Components

Token Embedding:

Convert discrete tokens to dense vectors:

$$e_i = W_e \times \text{token_id}_i$$

Where $W_e \in \mathbb{R}^{d_{model} \times vocab_size}$ is learned embedding matrix.

Positional Encoding:

Add position information (attention is permutation-invariant):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Absolute position encoding (original). Modern variations:

- Rotary position embedding (RoPE): Relative positions
- ALiBi: Attention with linear biases (no explicit embedding)

Combined Representation:

$$x_0 = e_i + PE_i$$

Where x_0 is input to first layer.

5.3 The Attention Mechanism

Attention computes weighted combination of values based on query-key similarity:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Interpretation:

- **Query:** What information does this position want?
- **Key:** What information do I have?
- **Value:** What do I output if keys match query?

Step-by-step:

1. Compute attention scores: $\text{scores} = QK^T/\sqrt{d_k}$ (shape: seq_len × seq_len)
2. Softmax: $\text{weights} = \text{softmax}(\text{scores})$ (normalize to probabilities)
3. Weighted sum: $\text{output} = \text{weights} \times V$

Example:

For query at position i, attention scores with all positions:

Query "the": [0.8, 0.1, 0.05, 0.02, 0.03] (scores for "the", "quick", "brown", "fox", "jumps")

After softmax: [0.55, 0.20, 0.10, 0.07, 0.08]

Output = 0.55×v_the + 0.20×v_quick + 0.10×v_brown + ... (weighted average of values)

Scaling Factor:

Divide by $\sqrt{d_k}$ to prevent softmax saturation:

$$\text{scores} = QK^T/\sqrt{d_k}$$

If not scaled, dot products grow large, softmax becomes one-hot (ignoring other positions).

6. Multi-Head Attention Mechanisms

6.1 Single vs Multi-Head Attention

Single-Head Problem:

Single attention mechanism only attends to one aspect of semantics.

Multi-Head Solution:

Multiple attention heads attend to different aspects simultaneously[^12].

$$\text{MultiHead}(Q, K, V) = \text{Concat}(h_1, \ldots, h_h)W_o$$

$$h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where:

- h: Number of heads (typically 8-16)
- W_i^Q, W_i^K, W_i^V: Head-specific projections
- W_o: Output projection

6.2 Attention Head Specialization

Different heads learn to attend to different aspects:

Head Type	amp; **What It Attends To**	amp; **Example**
Subject Head	amp; Long-range subject-verb relations	amp; "The cats ... jumped"
Adjacent Head	amp; Local context, adjacent tokens	amp; Neighboring words
Punctuation Head	amp; Punctuation-based dependencies	amp; Phrases separated by commas
Common Words Head	amp; High-frequency words	amp; Stop words for relational info
Rare Words Head	amp; Important specific words	amp; Named entities, keywords

Table 2: Typical Specialization of Attention Heads

Head Redundancy:

Many heads attend to similar patterns (heads are not fully independent). Recent research explores reducing head count without performance loss[^13].

6.3 Attention Head Computation

Step-by-step for Single Head:

Input: $x \in \mathbb{R}^{seq_len \times d_{model}}$ (sequence of embeddings)

1. **Project to Q, K, V:**
   $Q = xW^Q, \quad K = xW^K, \quad V = xW^V$
   Where $W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_k}$ (typically $d_k = d_{model}/h$)
2. **Compute attention scores:**
   $\text{scores} = \frac{QK^T}{\sqrt{d_k}}$
   Shape: (seq_len, seq_len)
3. **Mask (for decoder only):**
   Apply causal mask to prevent attending to future positions
4. **Softmax:**
   $\text{attn_weights} = \text{softmax}(\text{scores})$
   Each row sums to 1
5. **Apply to values:**
   $\text{output} = \text{attn_weights} \times V$
   Shape: (seq_len, d_k)

For Multiple Heads:

Repeat above for each head with different projections, then concatenate:

$$\text{MultiHead_output} = \text{Concat}(h_1, \ldots, h_h)W^o$$

6.4 Self-Attention vs Cross-Attention

Self-Attention:

Query, key, and value all from same source:

- Encoder: Attends to previous tokens in same sequence
- Decoder: Attends to previous generated tokens (causal)

Cross-Attention:

Query from one source, key/value from another (encoder-decoder models):

$$\text{CrossAttn} = \text{Attention}(Q_{\text{decoder}}, K_{\text{encoder}}, V_{\text{encoder}})$$

Allows decoder to focus on relevant parts of encoder output.

7. Stacking Attention Layers

7.1 Transformer Layer Stack

Transformers are built by stacking identical layers (typically 6-128 layers)[^14].

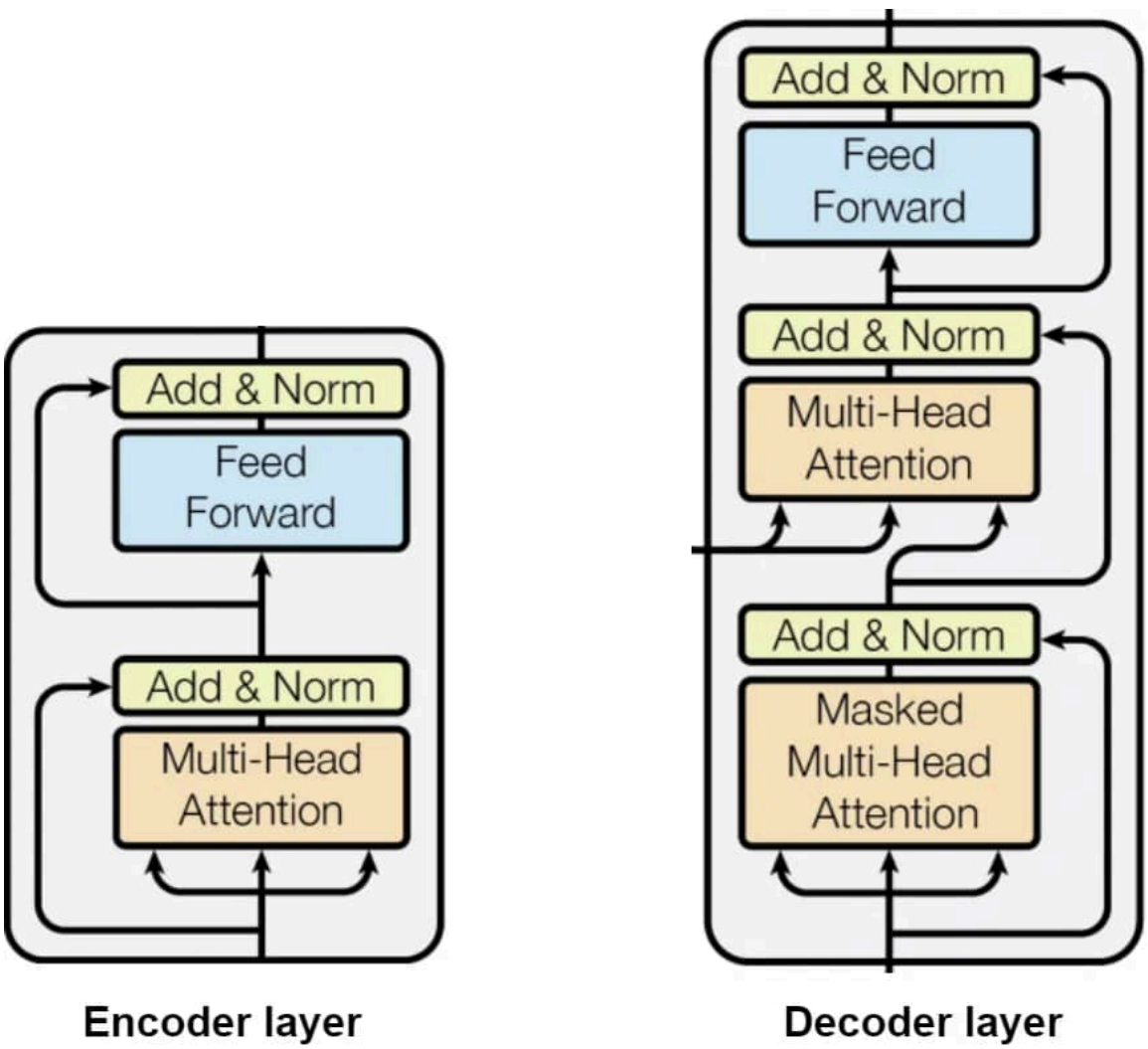

Figure 4: Stacked Transformer Layers: Information flows through multiple attention and feed-forward layers

Single Transformer Layer:

- Multi-head self-attention
- Residual connection + Layer normalization
- Feed-forward network (2 layers, hidden_dim ≈ 4× embedding_dim)
- Residual connection + Layer normalization

7.2 Information Flow Through Layers

Layer Dynamics:

- **Early layers:** Capture syntactic and local patterns (POS tags, phrases)
- **Middle layers:** Capture semantic and structural information (sentence structure, topics)
- **Late layers:** Refine representations for task-specific output (question answering, translation)

Depth vs Width Trade-off:

Dimension	amp; **Deeper (many layers)**	amp; **Wider (larger hidden)**
Parameters	amp; Moderate increase per layer	amp; Cubic growth in feed-forward
Compute	amp; Sequential (limits parallelism)	amp; Parallel within layer
Representational Power	amp; Better hierarchical abstraction	amp; Better within-layer expressivity
Optimization	amp; Harder (vanishing gradients)	amp; Easier (direct paths)
Typical Models	amp; GPT, LLaMA (70+ layers)	amp; BERT (12-24 layers)

Table 3: Depth vs Width Trade-offs

7.3 Residual Connections

Residual connections (skip connections) are critical for deep networks:

$$y = \text{Layer}(x) + x$$

Benefits:

- **Gradient flow:** Allows gradients to bypass layers, mitigating vanishing gradients
- **Information preservation:** Original information always reaches higher layers
- **Better optimization:** Easier optimization landscape with shortcuts

Mathematical Intuition:

Without residuals, gradient through deep network:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial x_1} \times \ldots \times \frac{\partial x_n}{\partial x}$$

Product of many small gradients → vanishing.

With residuals, one gradient path always includes identity:

$$\frac{\partial L}{\partial x} = \ldots + \frac{\partial L}{\partial y} \times 1$$

7.4 Layer Normalization

Layer normalization (LN) stabilizes training by normalizing activations:

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

Where:

- $\mu = \text{mean}(x)$: Average across hidden dimension

- $\sigma^2 = \text{var}(x)$: Variance across hidden dimension
- γ, β: Learned scale and shift parameters

Applied After Each Sub-layer:

Pre-norm architecture (modern, more stable):

$$x' = x + \text{Attention}(\text{LN}(x))$$
$$x'' = x' + \text{FFN}(\text{LN}(x'))$$

7.5 Feed-Forward Networks

After attention, each layer includes a feed-forward network (position-wise):

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Dimensions:

- Input: d_{model} (e.g., 768)
- Hidden: d_{ff} (e.g., 3072 = 4× input)
- Output: d_{model} (e.g., 768)

Purpose:

- Add non-linearity (ReLU activation)
- Increase model capacity
- Learned position-wise transformations

7.6 Complete Transformer Block

Pseudocode for single transformer layer:

```
x_input = x # Input: (seq_len, hidden_dim)

# Multi-head self-attention
attn_output = multi_head_attention(x_input)
x = x_input + dropout(attn_output) # Residual + dropout
x = layer_norm(x)

# Feed-forward
ff_output = ffn(x)
x = x + dropout(ff_output) # Residual + dropout
x = layer_norm(x)

# Output: (seq_len, hidden_dim)
```

Repeat this block N times (N = 12-128 in practice)

8. Text Generation with Transformers

8.1 Generation Process

Text generation is sequential token-by-token prediction using transformer models[^15].

Decoding Algorithm:

1. **Encode input** (if encoder-decoder, like T5)
2. **Initialize:** Start token (e.g., `<start>`)
3. **Loop until end-of-sequence or max length:**
   - Forward pass: Compute probability distribution for next token
   - Sample/select next token
   - Append to sequence
4. **Return:** Generated sequence

8.2 Decoding Strategies

Greedy Decoding:

Select highest probability token at each step:

$$w_t = \arg\max_w P(w_t | w_1, \ldots, w_{t-1})$$

```
def greedy_decode(model, prompt, max_length):
    tokens = prompt
    for _ in range(max_length):
        probs = model(tokens)[-1]  # Last token probabilities
        next_token = np.argmax(probs)
        tokens.append(next_token)
        if next_token == END_TOKEN:
            break
    return tokens
```

Disadvantages:

- Deterministic (same input → same output)
- Often leads to repetition
- Suboptimal (not searching for globally best sequence)

Beam Search:

Maintain k best partial sequences (beams):

1. Start with k copies of start token
2. At each step:
   - Expand each beam by all possible next tokens
   - Score combined sequences
   - Keep top k sequences
3. Stop when most beams reach end-of-sequence

Parameters:

- k (beam width): 1-10 typical (larger = better quality but slower)
- Length penalty: Prevent bias toward short sequences

```
def beam_search(model, prompt, beam_width=5, max_length=100):
    # Initialize beams: each is (sequence, log_prob)
    beams = [(prompt, 0.0)]

    for _ in range(max_length):
        candidates = []

        # Expand each beam
        for seq, score in beams:
            probs = model(seq)[-1]

            # Get top-k tokens
            for next_token, prob in top_k(probs, k=beam_width):
                new_seq = seq + [next_token]
                new_score = score + np.log(prob)
                candidates.append((new_seq, new_score))

        # Keep top-k sequences
        beams = sorted(candidates, key=lambda x: x[^1], reverse=True)[:beam_width]

    return beams[^0][^0]  # Best sequence
```

Advantages:

- Explores multiple hypotheses
- Better global sequence quality
- Still efficient (linear in length, exponential only in k)

8.3 Temperature and Sampling

High Temperature (T > 1):

Makes distribution flatter, more randomness:

$$P_{\text{scaled}}(w) = \frac{\exp(\log P(w)/T)}{\sum_w \exp(\log P(w)/T)}$$

Example: $P = [0.7, 0.2, 0.1]$ at $T = 1.0$

- At $T = 2.0$: $[0.52, 0.30, 0.18]$ (flatter, more diversity)

Low Temperature (T < 1):

Makes distribution sharper, more greedy:

- At $T = 0.5$: $[0.85, 0.10, 0.05]$ (more peaked)
- At $T \to 0$: Becomes greedy (highest prob only)

Top-k Sampling:

Only sample from top-k highest probability tokens:

```
def top_k_sample(probs, k=50, temperature=1.0):
    # Scale by temperature
    probs = probs ** (1 / temperature)
    probs = probs / probs.sum()

    # Zero out all but top-k
    top_indices = np.argsort(probs)[-k:]
    probs[~top_indices] = 0
    probs = probs / probs.sum()
```

```
    # Sample
    return np.random.choice(len(probs), p=probs)
```

Top-p (Nucleus) Sampling:

Sample from smallest set with cumulative probability > p:

```
def nucleus_sample(probs, p=0.95, temperature=1.0):
    probs = probs ** (1 / temperature)
    probs = probs / probs.sum()

    # Sort and compute cumulative sum
    sorted_indices = np.argsort(probs)[::-1]
    sorted_probs = probs[sorted_indices]
    cumsum = np.cumsum(sorted_probs)

    # Cutoff
    cutoff_idx = np.where(cumsum &gt; p)[^0][^0]
    cutoff_prob = sorted_probs[cutoff_idx]

    # Zero out below cutoff
    probs[probs &lt; cutoff_prob] = 0
    probs = probs / probs.sum()

    return np.random.choice(len(probs), p=probs)
```

9. Challenges and Optimization Techniques

9.1 Repetition and Diversity

Repetition Problem:

Models often generate repetitive text (e.g., "The the the...").

Causes:

- Beam search bias toward high-probability tokens
- Local maximum: continuing same pattern is locally optimal
- Sampling: repeating tokens have high probability

Solutions:

Diverse Beam Search:

Multiple beams forced to be diverse:

- Penalize similarity to previous beams
- Different sampling parameters per beam

No-Repeat N-gram:

Prevent repeating n-grams:

```
def no_repeat_ngram_decode(model, prompt, n=2, max_length=100):
    tokens = prompt
    seen_ngrams = set()

    for _ in range(max_length):
        probs = model(tokens)[-1]
```

```
        # Mask n-grams that appear in generated sequence
        for i in range(len(tokens) - n + 1):
            ngram = tuple(tokens[i:i+n-1])  # Last n-1 tokens
            if ngram in seen_ngrams:
                probs[tokens[i+n-1]] = 0

        # Sample
        next_token = sample(probs)
        tokens.append(next_token)

        # Add new n-grams
        if len(tokens) &gt;= n:
            seen_ngrams.add(tuple(tokens[-n:]))

    return tokens
```

9.2 Long-Range Dependencies

Challenge:

Transformers have $O(N^2)$ memory and compute for sequence length N.

Limitation:

Most transformers limited to 2048-4096 tokens (though recent work extends to 128k+).

Solutions:

Sparse Attention:

Limit attention to local neighborhood:

- Strided attention: Each position attends to every s-th position
- Fixed-size local window: Attend to local context window
- Combination: Local window + sparse strided

Efficient Attention:

- Flash Attention: IO-aware implementation reducing memory bandwidth
- Multi-query attention: Single key/value heads shared across queries
- Grouped-query attention: Balance between multi-query and multi-head

9.3 Hallucination and Factuality

Problem:

Models generate confident false statements.

Causes:

- Pre-training on web data (not all factual)
- No access to external knowledge during generation
- Training objective (next-token prediction) doesn't directly optimize for truthfulness

Solutions:

Retrieval Augmentation:

Provide model with relevant documents:

```
def rag_generation(query, documents):
    # Retrieve relevant documents
    relevant_docs = retrieve(query, documents)

    # Add to context
    context = summarize(relevant_docs)
    prompt = f"Context: {context}\n\nQuestion: {query}"

    # Generate grounded in context
    answer = model.generate(prompt)
    return answer
```

Knowledge Grounding:

Fine-tune on QA pairs with supporting evidence:

- Train to cite sources
- Penalize generation without evidence
- Use reward models to improve factuality

10. Hands-on Implementation Guide

10.1 Complete Query Autocomplete System

Step 1: Data Preparation

import pandas as pd
from sklearn.model_selection import train_test_split

Load query logs

queries_df = pd.read_csv("query_logs.csv")

Columns: user_id, query, timestamp, clicked, dwell_time

Create prefix-query pairs

def create_prefix_pairs(query):
pairs = []
for i in range(1, len(query) + 1):
prefix = query[:i]
pairs.append({"prefix": prefix, "query": query})
return pairs

prefix_data = []
for _, row in queries_df.iterrows():
pairs = create_prefix_pairs(row["query"])
for pair in pairs:
pair["clicked"] = row["clicked"]
prefix_data.append(pair)

df = pd.DataFrame(prefix_data)

Step 2: Feature Engineering

from collections import Counter
import numpy as np

Global popularity

query_popularity = df.groupby("query").size()
query_popularity = query_popularity / query_popularity.sum()

CTR

query_ctr = df[df["clicked"]].groupby("query").size() / df.groupby("query").size()

def extract_features(prefix, query):
features = {}

```
# Textual features
features["prefix_length"] = len(prefix)
features["query_length"] = len(query)
features["match_ratio"] = len(prefix) / len(query)

# Popularity features
features["query_popularity"] = query_popularity.get(query, 0)
features["query_ctr"] = query_ctr.get(query, 0)

# Recency (newer queries ranked higher)
features["recency_score"] = 1.0  # Would use timestamp in practice

# Diversity (avoid similar suggestions)
features["diversity_penalty"] = 0.0  # Computed at ranking time

return features
```

Step 3: Training Neural Ranker

import tensorflow as tf
from tensorflow.keras import layers

Create feature vectors

feature_vectors = []
labels = []

for _, row in df.iterrows():
features = extract_features(row["prefix"], row["query"])
feature_vectors.append(list(features.values()))
labels.append(1.0 if row["clicked"] else 0.0)

X = np.array(feature_vectors)
y = np.array(labels)

Split data

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

Neural ranker

```
model = tf.keras.Sequential([
layers.Dense(128, activation='relu'),
layers.Dropout(0.3),
layers.Dense(64, activation='relu'),
layers.Dropout(0.2),
layers.Dense(32, activation='relu'),
layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_split=0.1)
```

10.2 Next Word Prediction with Transformer

Step 1: Tokenization and Data Preparation

```
from transformers import AutoTokenizer
import torch
```

Load tokenizer

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

Sample text

```
text = """Natural language processing enables computers to understand...
[more text]
"""
```

Tokenize

```
tokens = tokenizer.encode(text)
```

Create training examples (sliding window)

```
def create_examples(tokens, window_size=10):
examples = []
for i in range(len(tokens) - window_size):
context = tokens[i:i+window_size]
target = tokens[i+window_size]
examples.append({"input_ids": context, "labels": target})
return examples

examples = create_examples(tokens, window_size=64)
```

Step 2: Build Language Model

```
import torch.nn as nn
from transformers import GPT2Model

class NextTokenPredictor(nn.Module):
def init(self, vocab_size=50257):
super().init()
```

self.embeddings = nn.Embedding(vocab_size, 768)
self.transformer = GPT2Model.from_pretrained("gpt2")
self.head = nn.Linear(768, vocab_size)

```
def forward(self, input_ids):
    x = self.transformer(input_ids)[^0]  # Last hidden state
    logits = self.head(x[:, -1, :])  # Last position
    return logits
```

model = NextTokenPredictor()

Step 3: Training

optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss()

for epoch in range(3):
for batch in dataloader:
input_ids = batch["input_ids"].to(device)
targets = batch["labels"].to(device)

```
    # Forward
    logits = model(input_ids)
    loss = criterion(logits, targets)

    # Backward
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Step 4: Generation

def generate_text(model, prompt_tokens, max_length=50):
generated = prompt_tokens.copy()

```
for _ in range(max_length):
    # Get next token probabilities
    with torch.no_grad():
        logits = model(torch.tensor([generated[-64:]]))

    # Top-p sampling
    probs = torch.softmax(logits, dim=-1)[^0]
    next_token = torch.multinomial(probs, 1).item()

    generated.append(next_token)

    if next_token == tokenizer.eos_token_id:
        break

return tokenizer.decode(generated)
```

10.3 Text Generation with Beam Search

def beam_search_generation(model, prompt, beam_width=5, max_length=100):
"""Generate text using beam search"""

```
# Initialize beams
prompt_tokens = tokenizer.encode(prompt)
```

```
beams = [(prompt_tokens, 0.0)]  # (sequence, cumulative_log_prob)

for step in range(max_length):
    candidates = []

    for sequence, cum_log_prob in beams:
        # Ensure sequence length ok
        if len(sequence) &gt; 2048:
            candidates.append((sequence, cum_log_prob))
            continue

        # Get next token probabilities
        with torch.no_grad():
            input_ids = torch.tensor([sequence[-1024:]])
            logits = model(input_ids)[0, -1, :]
            log_probs = torch.log_softmax(logits, dim=-1)

        # Top-k candidates
        top_log_probs, top_indices = torch.topk(log_probs, k=beam_width)

        for log_prob, idx in zip(top_log_probs, top_indices):
            new_sequence = sequence + [idx.item()]
            new_cum_log_prob = cum_log_prob + log_prob.item()
            candidates.append((new_sequence, new_cum_log_prob))

    # Keep top-k
    beams = sorted(candidates, key=lambda x: x[^1], reverse=True)[:beam_width]

    # Check if all ended
    if all(seq[-1] == tokenizer.eos_token_id for seq, _ in beams):
        break

# Return best sequence
best_sequence = beams[^0][^0]
return tokenizer.decode(best_sequence)
```

10.4 Production Serving

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class QueryRequest(BaseModel):
prefix: str
user_id: str

class TextGenerationRequest(BaseModel):
prompt: str
max_length: int = 100

Load models

qac_model = tf.keras.models.load_model("qac_ranker.h5")
generator = GPT2LMHeadModel.from_pretrained("gpt2")

@app.post("/autocomplete")
async def autocomplete(request: QueryRequest):
"""Query autocomplete endpoint"""

```
# Retrieve candidate queries
candidates = retrieve_candidates(request.prefix, top_k=100)

# Extract features
features = [extract_features(request.prefix, q) for q in candidates]
features_array = np.array([list(f.values()) for f in features])

# Score
scores = qac_model.predict(features_array).flatten()

# Rank
ranked = sorted(zip(candidates, scores), key=lambda x: x[^1], reverse=True)

return {"suggestions": [q for q, _ in ranked[:10]]}
```

@app.post("/generate")
async def generate(request: TextGenerationRequest):
"""Text generation endpoint"""

```
# Generate with beam search
output = beam_search_generation(
    generator,
    request.prompt,
    max_length=request.max_length
)

return {"generated_text": output}
```

Conclusion and Future Directions

This module has covered the practical implementation of LLMs across search, prediction, and generation. Key concepts include:

Key Takeaways:

- Word embeddings capture semantic relationships in dense vector spaces
- Query autocomplete combines traditional indexing with neural ranking for efficiency
- Transformers enable parallel processing and long-range dependencies
- Attention mechanisms allow models to focus on relevant information
- Text generation requires careful decoding strategies (greedy, beam search, sampling)
- Production systems require optimization for latency and scale

Emerging Directions:

- **Long Context:** Models supporting 100k-1M token contexts
- **Retrieval Integration:** Hybrid retrieval-generation for factuality
- **Efficient Inference:** Optimizations enabling deployment on mobile/edge
- **Multimodal Search:** Search combining text, image, and video
- **Personalization:** User-specific generation and ranking
- **Real-time Adaptation:** Models updating with user feedback

The field continues rapid evolution, with new architectures and techniques emerging constantly. Practitioners must stay current while maintaining deep understanding of fundamentals.

References

[1] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*[1].

[2] Radford, A., Narasimhan, K., Time, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. *OpenAI Blog*.

[3] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*[2].

[4] Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *EMNLP*, 14, 1532-1543.

[5] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[6] Shazeer, N., et al. (2020). Multi-query attention mechanisms for efficient large language model serving. *arXiv preprint arXiv:2305.13245*.

[7] Christiano, P. F., et al. (2023). Deep reinforcement learning from human preferences. *NeurIPS*, 29.

[8] Thawani, A. V., Singh, A., Sap, M., & Rashkin, H. (2021). Understanding attention and generalization in graph neural networks. *arXiv preprint arXiv:2104.13268*.

[9] Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. *ICML*.

[10] Choromanski, M. M., et al. (2020). Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*.

[^11] Kitaev, N., Kaiser, Ł., & Levskaya, A. (2020). Reformer: The efficient transformer. *ICLR*.

[^12] Raffel, C., et al. (2019). Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140), 1-67.

[^13] Lewis, M., et al. (2019). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*.

[^14] Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *NIPS*, 649-657.

[^15] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. *ACL*, 311-318.

⁂

1. https://dl.acm.org/doi/10.1145/3077136.3080758
2. https://www.amazon.science/publications/multi-objective-neural-retrieval-for-query-autocomplete
3. https://patents.google.com/patent/US20200104427A1/en
4. https://arxiv.org/html/2510.01574v1
5. https://www.sciencedirect.com/science/article/abs/pii/S1568494625014589
6. https://dergipark.org.tr/en/download/article-file/4579250
7. https://www.codecademy.com/article/transformer-architecture-self-attention-mechanism
8. https://www.ijcaonline.org/archives/volume187/number36/a-survey-of-query-refinement-techniques-from-neural-architectures-to-practical-applications/
9. https://ceur-ws.org/Vol-3633/paper14.pdf
10. https://www.datacamp.com/tutorial/how-transformers-work